Code Examples

Groups Project

Casey Stijlaart

Table of Contents

1. Docker Container

The docker container exists out of a dockerfile and two scripts. The two different scripts each serve their own purpose. The "install.sh" script has as purpose to install and update the necesarry packages and drivers. The second script, "start-script.sh" is the entrypoint script for the docker container and ensures the MongoDB is setup with the appropiate users and security.

Currently, the created docker image, from the dockerfile and script, is at V1.5 and includes various important aspects of the project. All the MongoDB drivers are installed, a MongoDB server is running on it, Google tests and code coverages trough lcov/gcov is included, and more.

As the docker image is a work in progress, due to changes and additions to the necesarry packages, yet the current version serves as a good starting point to extend upon.

1.1. Dockerfile

```
FROM mongo:6.0.4-jammy

COPY mongo.conf /mongo.conf

RUN apt-get update
RUN apt-get install -y \
    google-mock \
    wget \
    cmake \
    make \
    sudo

COPY install.sh /install.sh
RUN chmod +x /install.sh
RUN /install.sh

RUN apt-get -y --fix-broken install
EXPOSE 27017

COPY start-script.sh /start-script.sh
RUN chmod +x /start-script.sh

ENTRYPOINT ["sh", "-c", "sleep 10 && /start-script.sh"]
```

1.2. Scripts

1.2.1. install.sh

```
#!/bin/bash
```

```
apt-get update
apt-get install -y \
    libmongoc-dev \
    build-essential \
    devscripts \
    debian-keyring \
    debhelper \
    cmake \
    libssl-dev \
    pkg-config \
    python3-sphinx \
    zlib1g-dev \
    libicu-dev \
    libsasl2-dev \
    libsnappy-dev \
    libzstd-dev \
    libboost-dev \
    doxygen \
    libmongocrypt-dev \
    python3-sphinx \
    libutf8proc-dev \
    nlohmann-json3-dev \
    fakeroot \
    lcov \
    gcc \
    gcc-9

apt-get install gnupg curl

apt-get upgrade -y

apt-get update -y
apt-get install -y \
    libgtest-dev \
libgtest-dev

wget https://github.com/mongodb/mongo-c-driver/archive/refs/tags/1.24.4.tar.gz
tar -xzvf 1.24.4.tar.gz
cd mongo-c-driver-1.24.4
mkdir cmake-build
cd cmake-build
cmake -DBUILD_VERSION=1.24.4 ..
make
make install
cd ../..
rm -rf mongo-c-driver-1.24.4
rm 1.24.4.tar.gz

wget https://github.com/mongodb/mongo-cxx-driver/releases/download/r3.8.1/mongo-cxx-
driver-r3.8.1.tar.gz
tar -xzvf mongo-cxx-driver-r3.8.1.tar.gz
```

```
cd mongo-cxx-driver-r3.8.1
mkdir cmake-build
cd cmake-build
cmake -DCMAKE_INSTALL_PREFIX=/usr/local -DBUILD_VERSION=3.8.1 ..
make
make install
cd ../..
rm -rf mongo-cxx-driver-3.8.1
rm mongo-cxx-driver-r3.8.1.tar.gz

curl -fsSL https://pgp.mongodb.com/server-4.2.asc |
    sudo gpg -o /usr/share/keyrings/mongodb-server-4.2.gpg \
        --dearmor

echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-4.2.gpg ]
https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/4.2 multiverse" | sudo tee
/etc/apt/sources.list.d/mongodb-org-4.2.list
apt-get update
apt-get install -y mongodb-org

apt-get clean
rm -rf /var/lib/apt/lists/*
```

1.2.2. start-script.sh

```
#!/bin/bash

mongod --bind_ip_all --fork --logpath /var/log/mongodb.log

sleep 10

HOST_IP=$(getent hosts host.docker.internal | awk '{ print $1 }')

export SOURCE_HOST=$HOST_IP
export SOURCE_PORT=27017

MONGODB_URI="mongodb://${SOURCE_HOST}:${SOURCE_PORT}"

export MONGODB_URI

echo "MongoDB URI: $MONGODB_URI"

directory="/home/database/iot_data_lab_db"

if [ ! -d "$directory" ]; then
    mkdir -p "$directory"
fi

mongorestore --host $SOURCE_HOST --port $SOURCE_PORT --nsInclude='*.*' $directory
```

```
--drop

mongosh $MONGODB_URI <<EOF
use admin

if (!db.getUser('Administrator')) {
    db.createUser({
        user: 'Administrator',
        pwd: 'admin',
        roles: [
            { role: "userAdminAnyDatabase", db: "admin" },
            { role: "readWrite", db: "admin" }

        ]
    })
}
db.auth('Administrator','admin')
if (!db.getUser('guest')) {
    db.createUser({
        user: 'guest',
        pwd: 'guest',
        roles: [{ role: "read", db: "admin" }]
    })
}
EOF

mongod --shutdown --dbpath /data/db

# echo "security:
#   authorization: enabled" > /mongo.conf

mongod --config /mongo.conf --bind_ip_all --fork --logpath /var/log/mongodb.log

tail -f /dev/null
```

2. Database

The database has various aspects, to ensure its way of working. It is divided into three different classes, "Database", "DatabaseDataFormatter", "DatabaseManager". Each has their own purpose, ensuring lose coppeling and single responsibilty, SOLID.

2.1. Database Class

This class has the task to interact with the database server and its databases and collections. It ranges from adding users, to removing data. Overall the class implementation follows various design patterns such as Separation of Concerns and Dependency Injection.

```
#include "Database.hpp"
```

```
Database::Database(std::string connection_uri_input,
                   std::string db_storage_path)
    : connection_uri(connection_uri_input), db_path(db_storage_path) {
  static mongocxx::instance instance{};

  try {
    const auto api = mongocxx::options::server_api{
        mongocxx::options::server_api::version::k_version_1};
    client_options.server_api_opts(api);
    client = mongocxx::client(connection_uri, client_options);
  } catch (const std::exception& e) {
  }
}

bool Database::Begin() {
  try {
    const auto ping_cmd = bsoncxx::builder::basic::make_document(
        bsoncxx::builder::basic::kvp("ping", 1));
    auto admin_db = client["admin"];
    auto result = admin_db.run_command(ping_cmd.view());
    SignIn("guest", "guest");
  } catch (const std::exception& e) {
    return false;
  }

  return true;
}

bool Database::AddData(const std::string& data_input) {
  DatabaseDataFormatter format;

  if (!format.VerifyDataFormat(data_input)) {
    return false;
  }

  if (current_user == "" || current_user == "guest") {
    return false;
  }

  nlohmann::json data = format.CreateDataStructure(data_input);

  try {
    std::time_t now = std::time(nullptr);
    char formattedTime[9];
    std::strftime(formattedTime, sizeof(formattedTime), "%d%m%Y",
                  std::localtime(&now));

    database = client[current_user];
    mongocxx::collection coll = database[std::string(formattedTime)];
    bsoncxx::builder::basic::document document_builder;
```

```
    bsoncxx::document::value doc_value = bsoncxx::from_json(data.dump());
    bsoncxx::document::view data_view = doc_value.view();
    coll.insert_one(data_view);
    updateDatabase();
  } catch (const std::exception& e) {
    return false;
  }
  return true;
}

std::string Database::RetrieveDataCollection(const std::string& group_flag,
                                             const std::string& date) {
  if (current_user != group_flag) {
    if (!(GetAccessStatus(current_user, date))) {
      return "";
    }
  }

  try {
    database = client[group_flag];
    mongocxx::collection coll = database[date];

    mongocxx::cursor cursor =
        coll.find({}, mongocxx::options::find{}.projection(
                          bsoncxx::builder::basic::make_document(
                              bsoncxx::builder::basic::kvp("_id", 0))));

    nlohmann::json documents_array = nlohmann::json::array();

    for (auto doc : cursor) {
      nlohmann::json doc_json = nlohmann::json::parse(bsoncxx::to_json(doc));
      documents_array.push_back(doc_json);
    }

    return documents_array.dump(1);
  } catch (const std::exception& e) {
    return "[]";
  }
}

bool Database::EmptyCollection(const std::string& date) {
  if (current_user == "" || current_user == "guest") {
    return false;
  }

  try {
    database = client[current_user];
    mongocxx::collection collection = database[date];

    collection.drop();
    updateDatabase();
```

```
    return true;
  } catch (const std::exception& e) {
    std::cout << e.what() << std::endl;
    return false;
  }
}

bool Database::updateDatabase() {
  std::string cmd = "mongodump --uri " + connection_uri.to_string() +
                    " --out " + db_path + " --quiet > /dev/null 2>&1";

  int status = std::system(cmd.c_str());

  if (WIFEXITED(status) && WEXITSTATUS(status) != 0) {
    return false;
  }

  return true;
}

bool Database::AddUser(const std::string& user_group, const std::string& pwd) {
  if (current_user != "Administrator") {
    return false;
  }

  try {
    auto admin_db = client["admin"];
    auto user_doc = bsoncxx::builder::basic::make_document(
        bsoncxx::builder::basic::kvp("createUser", user_group),
        bsoncxx::builder::basic::kvp("pwd", pwd),
        bsoncxx::builder::basic::kvp(
            "roles", bsoncxx::builder::basic::make_array(
                        bsoncxx::builder::basic::make_document(
                            bsoncxx::builder::basic::kvp("role", "readWrite"),
                            bsoncxx::builder::basic::kvp("db", user_group)),
                        bsoncxx::builder::basic::make_document(
                            bsoncxx::builder::basic::kvp("role", "read"),
                            bsoncxx::builder::basic::kvp("db", "admin"),
                            bsoncxx::builder::basic::kvp("collection",
                                                         "system.users")))));
    auto result = admin_db.run_command(user_doc.view());
    if (!(result.view()["ok"].get_double() == 1.0)) {
      return false;
    }

  } catch (const std::exception& e) {
    std::cout << e.what() << std::endl;
    return false;
  }
  updateDatabase();
  return true;
```

```
}

bool Database::RemoveUser(const std::string& user_group) {
  if (current_user != "Administrator") {
    return false;
  }

  try {
    auto admin_db = client["admin"];
    auto result = admin_db.run_command(bsoncxx::builder::basic::make_document(
        bsoncxx::builder::basic::kvp("dropUser", user_group)));

    if (!(result.view()["ok"].get_double() == 1.0)) {
      return false;
    }

  } catch (const std::exception& e) {
    std::cout << e.what() << std::endl;
    return false;
  }
  return true;
}

bool Database::GetAccessStatus(const std::string& user_group,
                               const std::string& col_name) {
  if (!ExistingUser(user_group)) {
    return false;
  }

  try {
    auto admin_db = client["admin"];
    auto result = admin_db.run_command(bsoncxx::builder::basic::make_document(
        bsoncxx::builder::basic::kvp("usersInfo", user_group)));

    auto roles_array = result.view()["users"][0]["roles"].get_array().value;

    for (auto&& role : roles_array) {
      std::string role_db = role["db"].get_string().value.to_string();
      if (role_db == current_user) {
        return true;
      }
    }
  } catch (const std::exception& e) {
    std::cout << e.what() << std::endl;
    return false;
  }
  return false;
}

std::string Database::ActiveUser() { return GetCurrentUser(); }
```

```
bool Database::SignIn(const std::string& username, const std::string& pwd) {
  std::string temp_con_uri = GenerateConnectionUri(username, pwd);

  if (temp_con_uri == "") {
    return false;
  }

  connection_uri = mongocxx::uri(temp_con_uri);

  try {
    auto db_admin = client["admin"];
    client = mongocxx::client(connection_uri, client_options);
    const auto ping_cmd = bsoncxx::builder::basic::make_document(
        bsoncxx::builder::basic::kvp("ping", 1));
    auto db = client[username];
    auto result = db.run_command(ping_cmd.view());

    if (!(result.view()["ok"].get_double() == 1.0)) {
      return false;
    }
    current_user = username;
  } catch (const std::exception& e) {
    std::cout << "Sign in Error : " << e.what() << std::endl;
    return false;
  }
  return true;
}

bool Database::SignOut() {
  if (!(SignIn("guest", "guest"))) {
    return false;
  }
  return true;
}

bool Database::AllowAccess(const std::string& allowed_group,
                           const std::string& collection) {
  try {
    auto admin_db = client["admin"];
    bool access_user = GetAccessStatus(allowed_group, collection);

    if (access_user) {
      return true;
    }

    if (ExistingUser(allowed_group)) {
      bsoncxx::builder::basic::array new_roles;

      auto current_roles_result =
          admin_db.run_command(bsoncxx::builder::basic::make_document(
              bsoncxx::builder::basic::kvp("usersInfo", allowed_group)));
```

```
      auto current_roles =
          current_roles_result.view()["users"][0]["roles"].get_array().value;

      for (auto&& role : current_roles) {
        new_roles.append(role.get_value());
      }

      if (collection == "All") {
        new_roles.append(bsoncxx::builder::basic::make_document(
            bsoncxx::builder::basic::kvp("role", "read"),
            bsoncxx::builder::basic::kvp("db", current_user)));
      } else {
        if (!ExistingCollection(collection)) {
          return false;
        }
        new_roles.append(bsoncxx::builder::basic::make_document(
            bsoncxx::builder::basic::kvp("role", "read"),
            bsoncxx::builder::basic::kvp("db", current_user),
            bsoncxx::builder::basic::kvp("collection", collection)));
      }

      auto command = bsoncxx::builder::basic::make_document(
          bsoncxx::builder::basic::kvp("updateUser", allowed_group),
          bsoncxx::builder::basic::kvp("roles", new_roles),
          bsoncxx::builder::basic::kvp(
              "writeConcern", bsoncxx::builder::basic::make_document()));

      auto result = admin_db.run_command(command.view());

      if (result.view()["ok"].get_double() != 1.0) {
        return false;
      }
    } else {
      return false;
    }
  } catch (const std::exception& e) {
    std::cout << " exception" << std::endl;
    std::cout << e.what() << std::endl;
    return false;
  }
  updateDatabase();
  return true;
}

bool Database::ExistingCollection(const std::string& collection) {
  auto db = client[current_user];
  auto collections = db.list_collection_names();

  for (auto&& collection_check : collections) {
    if (collection_check == collection) {
```

```
      return true;
    }
  }
  return false;
}

std::string Database::GetCurrentUser() { return current_user; }

bool Database::ExistingUser(const std::string& username) {
  auto admin_db = client["admin"];
  auto result_exist =
      admin_db.run_command(bsoncxx::builder::basic::make_document(
          bsoncxx::builder::basic::kvp("usersInfo", username)));

  if (result_exist.view()["ok"].get_double() != 1.0) {
    return false;
  }
  return true;
}

std::string Database::GenerateConnectionUri(const std::string& username,
                                            const std::string pwd) {
  size_t delimiterPos = connection_uri.to_string().find("://");
  size_t atSymbolPos = connection_uri.to_string().find("@");
  std::string temp_connection_uri;

  if (delimiterPos != std::string::npos) {
    std::string firstPart =
        connection_uri.to_string().substr(0, delimiterPos + 3);
    std::string secondPart = connection_uri.to_string().substr(atSymbolPos + 1);
    temp_connection_uri = firstPart + username + ":" + pwd + "@" + secondPart;
    return temp_connection_uri;
  } else {
    return "";
  }
}
```

2.2. DatabaseDataFormatter Class

This class has the purpose of creating the correct format for data to be inserted into the database. As it is of great important for data to be consistent, and besides to be in json, as this is expected from the database.

```
#include "DatabaseDataFormatter.hpp"

nlohmann::json DatabaseDataFormatter::CreateDataStructure(
    const std::string& data_input) {
  try {
    std::time_t now = std::time(nullptr);
```

```
    char formattedTime[50];
    std::strftime(formattedTime, sizeof(formattedTime), "%Y-%m-%d %H:%M:%S",
                  std::localtime(&now));

    nlohmann::json jsonData = nlohmann::json::parse(data_input);

    nlohmann::json data_entry = {{"timestamp", formattedTime},
                                 {"data", jsonData}};

    return data_entry;
  } catch (const nlohmann::json::exception& e) {
    return {};
  }
}

bool DatabaseDataFormatter::VerifyDataFormat(const std::string& data) {
  try {
    nlohmann::json jsonData = nlohmann::json::parse(data);
  } catch (const std::exception& e) {
    return false;
  }
  return true;
}
```

2.3. DatabaseManager Class

Lastly there is the manager class. This class serves as the in between layer, between the actual main application and the database classes. As there are various commands to be processed and linked to correct functionality, this class is created. This ensures that there is a lose coppeling between the main application and the actual checking of the commands and linking the functionality. This class ensures that the main application doesnt need to worry about all the various commands that can occur, but rather can just receive messages and give it to the corresponding manager. Besides that, when it comes to the actual class implementation, it makes use of the strategy pattern, allowing for easy extension when new commands are added. Overall the implementation ensures encapsulation, Dependency Injection and separation of concerns.

```
#include "DatabaseManager.hpp"

DatabaseManager::DatabaseManager(IDatabase* database_input)
    : database(database_input) {}

bool DatabaseManager::Begin() {
  if (!(database->Begin())) {
    return false;
  }
  return true;
}

std::vector<std::string> DatabaseManager::HandleEvent(
```

```
    const nlohmann::json& data_input) {
  std::vector<std::string> result;

  if (data_input.empty() || !data_input.contains("command")) {
    result.push_back("Error: Invalid or missing command in the input.");
    return result;
  }

  std::string command = data_input["command"];

  if (command == Commands::Database::SignIn) {
    return HandleSignIn(data_input);
  } else if (command == Commands::Database::SignOut) {
    return HandleSignOut();
  } else if (command == Commands::Database::RemoveUser) {
    return HandleRemoveUser(data_input);
  } else if (command == Commands::Database::AddUser) {
    return HandleAddUser(data_input);
  } else if (command == Commands::Database::AllowAccess) {
    return HandleAllowAccess(data_input);
  } else if (command == Commands::Database::GetAccessStatus) {
    return HandleGetAccessStatus(data_input);
  } else if (command == Commands::Database::GetActiveUser) {
    return HandleGetActiveUser();
  } else if (command == Commands::Database::AddData) {
    return HandleAddData(data_input);
  } else if (command == Commands::Database::RetrieveDataCollection) {
    return HandleRetrieveDataCollection(data_input);
  } else if (command == Commands::Database::EmptyCollection) {
    return HandleEmptyCollection(data_input);
  } else {
    result.push_back("ERROR");
  }
  return result;
}

std::vector<std::string> DatabaseManager::HandleSignIn(
    const nlohmann::json& data) {
  std::vector<std::string> result;

  if (data.contains("data") && data["data"].is_object()) {
    nlohmann::json dataFields = data["data"];
    if (dataFields.contains("username") && dataFields.contains("password")) {
      std::string username = dataFields["username"];
      std::string password = dataFields["password"];
      if (database->SignIn(username, password)) {
        result.push_back("OK");
      } else {
        result.push_back("ERROR");
      }
    } else {
```

```
      result.push_back("ERROR");
    }
  } else {
    result.push_back("ERROR");
  }

  return result;
}

std::vector<std::string> DatabaseManager::HandleSignOut() {
  std::vector<std::string> result;

  if (database->SignOut()) {
    result.push_back("OK");
  } else {
    result.push_back("ERROR");
  }

  return result;
}

std::vector<std::string> DatabaseManager::HandleRemoveUser(
    const nlohmann::json& data) {
  std::vector<std::string> result;

  if (data.contains("data") && data["data"].is_object()) {
    nlohmann::json dataFields = data["data"];
    if (dataFields.contains("username")) {
      std::string username = dataFields["username"];
      if (database->RemoveUser(username)) {
        result.push_back("OK");
      } else {
        result.push_back("ERROR");
      }
    } else {
      result.push_back("ERROR");
    }
  } else {
    result.push_back("ERROR");
  }
  return result;
}

std::vector<std::string> DatabaseManager::HandleAddUser(
    const nlohmann::json& data) {
  std::vector<std::string> result;

  if (data.contains("data") && data["data"].is_object()) {
    nlohmann::json dataFields = data["data"];
    if (dataFields.contains("username") && dataFields.contains("password")) {
      std::string username = dataFields["username"];
```

```
      std::string password = dataFields["password"];
      if (database->AddUser(username, password)) {
        result.push_back("OK");
      } else {
        result.push_back("ERROR");
      }
    } else {
      result.push_back("ERROR");
    }
  } else {
    result.push_back("ERROR");
  }

  return result;
}

std::vector<std::string> DatabaseManager::HandleAllowAccess(
    const nlohmann::json& data) {
  std::vector<std::string> result;

  if (data.contains("data") && data["data"].is_object()) {
    nlohmann::json dataFields = data["data"];
    if (dataFields.contains("username") && dataFields.contains("collection")) {
      std::string username = dataFields["username"];
      std::string collection = dataFields["collection"];
      if (database->AllowAccess(username, collection)) {
        result.push_back("OK");
      } else {
        result.push_back("ERROR");
      }
    } else {
      result.push_back("ERROR");
    }
  } else {
    result.push_back("ERROR");
  }
  return result;
}

std::vector<std::string> DatabaseManager::HandleGetAccessStatus(
    const nlohmann::json& data) {
  std::vector<std::string> result;

  if (data.contains("data") && data["data"].is_object()) {
    nlohmann::json dataFields = data["data"];
    if (dataFields.contains("username") && dataFields.contains("collection")) {
      std::string username = dataFields["username"];
      std::string collection = dataFields["collection"];
      if (database->GetAccessStatus(username, collection)) {
        result.push_back("OK");
      } else {
```

```
        result.push_back("ERROR");
      }
    } else {
      result.push_back("ERROR");
    }
  } else {
    result.push_back("ERROR");
  }

  return result;
}

std::vector<std::string> DatabaseManager::HandleGetActiveUser() {
  std::vector<std::string> result;
  std::string ret = database->ActiveUser();
  if (ret == "") {
    result.push_back("ERROR");
  } else {
    result.push_back(ret);
  }
  return result;
}

std::vector<std::string> DatabaseManager::HandleAddData(
    const nlohmann::json& data) {
  std::vector<std::string> result;

  if (data.contains("data") && data["data"].is_object()) {
    nlohmann::json dataFields = data["data"];

    std::string jsonDataString = dataFields.dump();
    if (database->AddData(jsonDataString)) {
      result.push_back("OK");
    } else {
      result.push_back("ERROR");
    }
  } else {
    result.push_back("ERROR");
  }

  return result;
}

std::vector<std::string> DatabaseManager::HandleRetrieveDataCollection(
    const nlohmann::json& data) {
  std::vector<std::string> result;

  if (data.contains("data") && data["data"].is_object()) {
    nlohmann::json dataFields = data["data"];
    if (dataFields.contains("username") && dataFields.contains("collection")) {
      std::string username = dataFields["username"];
```

```
      std::string collection = dataFields["collection"];
      std::string data_ret =
          database->RetrieveDataCollection(username, collection);
      if (data_ret != "") {
        result.push_back(data_ret);
      } else {
        result.push_back("ERROR");
      }
    } else {
      result.push_back("ERROR");
    }
  } else {
    result.push_back("ERROR");
  }
  return result;
}

std::vector<std::string> DatabaseManager::HandleEmptyCollection(
    const nlohmann::json& data) {
  std::vector<std::string> result;

  if (data.contains("data") && data["data"].is_object()) {
    nlohmann::json dataFields = data["data"];
    if (dataFields.contains("collection")) {
      std::string collection = dataFields["collection"];
      if (database->EmptyCollection(collection)) {
        result.push_back("OK");
      } else {
        result.push_back("ERROR");
      }
    } else {
      result.push_back("ERROR");
    }
  } else {
    result.push_back("ERROR");
  }

  return result;
}
```

2.4. Testing

While the implementation of various classes are important, the testing of the software trough e.g. unit testing, has a great importance as well. To show case how unit testing is done within our project, the test cases of the DatabaseManager is given. As the DatabaseManager makes use of the database trough dependency injection, the database could simply be mocked to test the DatabaseManager's functionality itself.

```
#include <gmock/gmock.h>
```

```
#include <gtest/gtest.h>

#include "DatabaseManager.hpp"
#include "MockIDatabase.hpp"

class DatabaseManagerTest : public ::testing::Test {
 protected:
  void SetUp() override {
    mockDatabase = new MockDatabase();
    databaseManager = new DatabaseManager(mockDatabase);
  }

  void TearDown() override {
    delete databaseManager;
    delete mockDatabase;
  }

  MockDatabase* mockDatabase;
  DatabaseManager* databaseManager;
};

TEST_F(DatabaseManagerTest, HandleEventErrorMissingCommand) {
  nlohmann::json invalidData = {{"invalid_key", "value"}};
  auto result = databaseManager->HandleEvent(invalidData);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "Error: Invalid or missing command in the input.");
}

TEST_F(DatabaseManagerTest, HandleEventErrorInvalidCommand) {
  nlohmann::json invalidData = {{"command", "invalid"}};
  auto result = databaseManager->HandleEvent(invalidData);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, BeginSuccess) {
  EXPECT_CALL(*mockDatabase, Begin())
      .WillOnce(::testing::Return(true));
  auto result = databaseManager->Begin();
  ASSERT_TRUE(result);
}

TEST_F(DatabaseManagerTest, BeginErrorMissingParameter) {
  EXPECT_CALL(*mockDatabase, Begin())
      .WillOnce(::testing::Return(false));
  auto result = databaseManager->Begin();
  ASSERT_FALSE(result);
}

TEST_F(DatabaseManagerTest, HandleSignInSuccess) {
  nlohmann::json validSignInData = {
```

```
      {"command", "SignIn"},
      {"data", {{"username", "test_user"}, {"password", "test_password"}}}};
  EXPECT_CALL(*mockDatabase, SignIn("test_user", "test_password"))
      .WillOnce(::testing::Return(true));
  auto result = databaseManager->HandleEvent(validSignInData);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "OK");
}

TEST_F(DatabaseManagerTest, HandleSignInErrorDatabase) {
  nlohmann::json validSignInData = {
      {"command", "SignIn"},
      {"data", {{"username", "test_user"}, {"password", "test_password"}}}};
  EXPECT_CALL(*mockDatabase, SignIn("test_user", "test_password"))
      .WillOnce(::testing::Return(false));
  auto result = databaseManager->HandleEvent(validSignInData);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, HandleSignInBeginMissingParameter) {
  nlohmann::json data = {{"command", "SignIn"},
                         {"data", {{"username", "test_user"}}}};
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, HandleSignOutSucces) {
  nlohmann::json data = {
      {"command", "SignOut"},
  };
  EXPECT_CALL(*mockDatabase, SignOut()).WillOnce(::testing::Return(true));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "OK");
}

TEST_F(DatabaseManagerTest, HandleSignOutErrorDatabase) {
  nlohmann::json data = {
      {"command", "SignOut"},
  };
  EXPECT_CALL(*mockDatabase, SignOut()).WillOnce(::testing::Return(false));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, RemoveUserSucces) {
  nlohmann::json data = {{"command", "RemoveUser"},
                         {"data", {{"username", "test_user"}}}};
```

```
  std::string expectedUsername = data["data"]["username"];
  EXPECT_CALL(*mockDatabase, RemoveUser(expectedUsername))
      .WillOnce(::testing::Return(true));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "OK");
}

TEST_F(DatabaseManagerTest, RemoveUserErrorDatabase) {
  nlohmann::json data = {{"command", "RemoveUser"},
                         {"data", {{"username", "test_user"}}}};
  std::string expectedUsername = data["data"]["username"];
  EXPECT_CALL(*mockDatabase, RemoveUser(expectedUsername))
      .WillOnce(::testing::Return(false));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, RemoveUserErrorInvalidUser) {
  nlohmann::json data = {{"command", "RemoveUser"},
                         {"data", {{"username", "invalid"}}}};
  std::string expectedUsername = data["data"]["username"];
  EXPECT_CALL(*mockDatabase, RemoveUser(expectedUsername))
      .WillOnce(::testing::Return(false));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, AddUserSucces) {
  nlohmann::json data = {
      {"command", "AddUser"},
      {"data", {{"username", "test_user"}, {"password", "test_password"}}}};

  EXPECT_CALL(*mockDatabase, AddUser("test_user", "test_password"))
      .WillOnce(::testing::Return(true));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "OK");
}

TEST_F(DatabaseManagerTest, AddUserErrorDatabase) {
  nlohmann::json data = {
      {"command", "AddUser"},
      {"data", {{"username", "test_user"}, {"password", "test_password"}}}};

  EXPECT_CALL(*mockDatabase, AddUser("test_user", "test_password"))
      .WillOnce(::testing::Return(false));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
```

```
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, AddUserErrorMissingParameter) {
  nlohmann::json data = {{"command", "AddUser"},
                         {"data", {{"username", "invalid"}}}};
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, AllowAccessSucces) {
  nlohmann::json data = {
      {"command", "AllowAccess"},
      {"data", {{"username", "test_user"}, {"collection", "10102023"}}}};
  EXPECT_CALL(*mockDatabase, AllowAccess("test_user", "10102023"))
      .WillOnce(::testing::Return(true));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "OK");
}

TEST_F(DatabaseManagerTest, AllowAccessErrorDatabase) {
  nlohmann::json data = {
      {"command", "AllowAccess"},
      {"data", {{"username", "test_user"}, {"collection", "10102023"}}}};
  EXPECT_CALL(*mockDatabase, AllowAccess("test_user", "10102023"))
      .WillOnce(::testing::Return(false));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, AllowAccessErrorMissingParameter) {
  nlohmann::json data = {{"command", "AllowAccess"},
                         {"data", {{"username", "test_user"}}}};
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, GetAccessStatusSucces) {
  nlohmann::json data = {
      {"command", "GetAccessStatus"},
      {"data", {{"username", "test_user"}, {"collection", "10102023"}}}};

  EXPECT_CALL(*mockDatabase, GetAccessStatus("test_user", "10102023"))
      .WillOnce(::testing::Return(true));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "OK");
```

```
}

TEST_F(DatabaseManagerTest, GetAccessStatusErrorDatabase) {
  nlohmann::json data = {
      {"command", "GetAccessStatus"},
      {"data", {{"username", "test_user"}, {"collection", "10102023"}}}};

  EXPECT_CALL(*mockDatabase, GetAccessStatus("test_user", "10102023"))
      .WillOnce(::testing::Return(false));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, GetAccessStatusErrorMissingParameter) {
  nlohmann::json data = {{"command", "GetAccessStatus"},
                         {"data", {{"username", "test_user"}}}};
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, GetActiveUserSucces) {
  nlohmann::json data = {{"command", "GetActiveUser"}};
  EXPECT_CALL(*mockDatabase, ActiveUser())
      .WillOnce(::testing::Return("test_user"));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "test_user");
}

TEST_F(DatabaseManagerTest, GetActiveUserErrorDatabase) {
  nlohmann::json data = {{"command", "GetActiveUser"}};
  EXPECT_CALL(*mockDatabase, ActiveUser())
      .WillOnce(::testing::Return(""));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, AddDataSucces) {
  nlohmann::json data = {{"command", "AddData"},
                         {"data", {{"temperature", "12"}}}};
  EXPECT_CALL(*mockDatabase, AddData("{\"temperature\":\"12\"}"))
      .WillOnce(::testing::Return(true));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "OK");
}

TEST_F(DatabaseManagerTest, AddDataErrorDatabase) {
```

```
  nlohmann::json data = {{"command", "AddData"},
                         {"data", {{"temperature", "12"}}}};
  EXPECT_CALL(*mockDatabase, AddData("{\"temperature\":\"12\"}"))
      .WillOnce(::testing::Return(false));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, AddDataErrorNoData) {
  nlohmann::json data = {{"command", "AddData"}};
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, RetrieveDataCollectionSuccess) {
  nlohmann::json data = {
      {"command", "RetrieveDataCollection"},
      {"data", {{"username", "test_user"}, {"collection", "10102023"}}}};
  std::string expectedData = R"({"temp": 12, "humid": 67})";
  EXPECT_CALL(*mockDatabase, RetrieveDataCollection("test_user", "10102023"))
      .WillOnce(::testing::Return(expectedData));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], expectedData);
}

TEST_F(DatabaseManagerTest, RetrieveDataCollectionErrorDatabase) {
  nlohmann::json data = {
      {"command", "RetrieveDataCollection"},
      {"data", {{"username", "test_user"}, {"collection", "10102023"}}}};
  EXPECT_CALL(*mockDatabase, RetrieveDataCollection("test_user", "10102023"))
      .WillOnce(::testing::Return(""));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, RetrieveDataCollectionErrorMissingParameter) {
  nlohmann::json data = {{"command", "RetrieveDataCollection"},
                         {"data", {{"username", "test_user"}}}};

  auto result = databaseManager->HandleEvent(data);

  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, EmptyCollectionSuccess) {
  nlohmann::json data = {{"command", "EmptyCollection"},
```

```
                        {"data", {{"collection", "10102023"}}}};
  EXPECT_CALL(*mockDatabase, EmptyCollection("10102023"))
      .WillOnce(::testing::Return(true));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "OK");
}

TEST_F(DatabaseManagerTest, EmptyCollectionErrorDatabase) {
  nlohmann::json data = {{"command", "EmptyCollection"},
                        {"data", {{"collection", "10102023"}}}};
  EXPECT_CALL(*mockDatabase, EmptyCollection("10102023"))
      .WillOnce(::testing::Return(false));
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}

TEST_F(DatabaseManagerTest, EmptyCollectionErrorMissingParameter) {
  nlohmann::json data = {{"command", "EmptyCollection"}};
  auto result = databaseManager->HandleEvent(data);
  ASSERT_EQ(result.size(), 1);
  ASSERT_EQ(result[0], "ERROR");
}
```